Spatial Networks

Analyzing spatial networks, for instance, road networks or infrastructural networks, is a popular area of geospatial sciences, providing insights into urban planning, transportation systems, power grid optimization, and many others. This chapter explores methods and techniques for obtaining, transforming, analyzing, and visualizing spatial networks using dedicated Python libraries, including OSMnx, GeoPandas, and NetworkX. As our primary example, we will analyze road networks, which are prototypical spatial networks in which both the nodes (road intersections) and the edges (road segments) are embedded in a geographical space.

We will start by downloading and converting road networks from OpenStreetMap (OSM) into GeoPandas GeoDataFrames and then transforming these GeoDataFrames back into spatial networks. Following this, we will explore methods for visualizing spatial networks to understand their structure and connectivity. Then, we will conduct network-analytic steps that are particularly useful for spatial networks, such as computing and visualizing the shortest paths between nodes and generating walking-distance isochrones. Finally, we will obtain and interpret network statistics, calculate centrality measures, and perform community detection to identify clusters within the network.

Chapter outline

81. Road Networks in GeoPandas

82. From GeoDataFrames to Spatial Networks

83. Visualizing Spatial Networks

84. Calculating Shortest Paths

85. Visualizing the Shortest Path

86. Generating Walking Distance Isochrones

87. Obtaining Spatial Network Statistics

88. Network Centrality Measures

89. Community Detection in Spatial Networks

Summary on Spatial Networks

81. Road Networks in GeoPandas

Our primary example of spatial networks will be road networks, which provide insights into urban planning, transportation systems, and many other use cases of spatial analytics. As a first technical step, we review how to download road networks from OpenStreetMap and store them in GeoDataFrames, enabling more detailed spatial analysis and visualization using the powerful capabilities of GeoPandas.

We will start by downloading the road network within a specified administrative boundary using `osmnx` and the name of the target area as input. As an example, we will use the 1st district of Budapest and focus on the road network. Then, using the `graph_from_polygon` function, we query the road network, and with the help of the `graph_to_gdfs` function, we project the graph object into separate GeoDataFrames for nodes and edges. Finally, we print out basic information about the network we obtained.

In [1]:
```
# Import osmnx for geospatial data from OpenStreetMap
import osmnx as ox

# Define the polygon forthe 1st district of Budapest
admin_district = ox.geocode_to_gdf('1st district, Budapest')
admin_poly = admin_district.geometry.values[0]

# Download the road network for driving within the admin boundary
G = ox.graph_from_polygon(admin_poly, network_type='drive')

# Print the number of nodes and edges in the road network
print("Number of nodes in the original graph:", G.number_of_nodes())
print("Number of edges in the original graph:", G.number_of_edges())
print()

# Convert the network graph to GeoDataFrames for nodes and edges
nodes, edges = ox.graph_to_gdfs(G)

# Display the first few rows of the nodes GeoDataFrame
display(nodes.head(3))

# Display the first few rows of the edges GeoDataFrame
display(edges.head(3))


# Print the features stored in each GeoDataFrame
print("Keys in the nodes table:", list(nodes.keys()))
print("Keys in the edges table:", list(edges.keys()))
print()

# Print the total number of nodes and edges
print("Number of nodes in the node table:", len(nodes))
print("Number of edges in the node table:", len(edges))
```

```
Number of nodes in the original graph: 397
Number of edges in the original graph: 781
```

osmid	y	x	street_count	highway	geometry
35500835	47.490117	19.045621	4	NaN	POINT (19.04562 47.49012)
35523878	47.491303	19.045614	4	NaN	POINT (19.04561 47.4913)
35523895	47.498558	19.040555	3	NaN	POINT (19.04055 47.49856)

u	v	key	osmid	highway	lanes	maxspeed	oneway	ref	re
35500835	**1676318037**	**0**	[464432632, 8107826]	primary	3	30	True	3	
	1676318015	**0**	[1451392003, 292642956, 56066399]	secondary	2	[50, 30]	True	NaN	
35523878	**10921844764**	**0**	[24255970, 56066397]	secondary	[2, 1]	50	True	NaN	

```
Keys in the nodes table: ['y', 'x', 'street_count', 'highway', 'geometry']
Keys in the edges table: ['osmid', 'highway', 'lanes', 'maxspeed', 'onewa
y', 'ref', 'reversed', 'length', 'geometry', 'name', 'width', 'junction',
'bridge', 'tunnel', 'access']

Number of nodes in the node table: 397
Number of edges in the node table: 781
```

The output of the previous cell provides detailed information about the road network in Budapest's 1st district. For instance, it shows that the road network used for driving contains 397 nodes and 781 edges, both in the queried graph and in the transformed node-and-edge table.

The detailed queries to print the features stored in these tables reveal additional information in the OSM data, such as geometries, OSM IDs, and road segment lengths, allowing us to browse and use various types of information describing these spatial networks.

82. From GeoDataFrames to Spatial Networks

After turning a road network graph into GeoDataFrames, we are now doing the reverse operation by transforming GeoDataFrames into spatial networks. This step,

complementing the previous one, enables us to leverage graph theory for transportation planning, network optimization, and other use cases. By converting the node and edge tables stored in GeoPandas GeoDataFrames into a NetworkX graph, we will be able to perform a wide range of network analytics exercises, as we will learn later on in this chapter.

In this section, we will demonstrate how to convert the edges of a road network GeoDataFrame into a `networkx` graph. Between two intersections, there may be multiple links, such as driving lanes. We use the `nx.MultiGraph` object for efficient data storage. This process involves iterating through the `edges` GeoDataFrame and adding each edge to the NetworkX graph with appropriate attributes, such as the length of each road segment. We also extract information from the `nodes` table, as we include the node locations as coordinate pairs to ensure the resulting graph contains the relevant spatial information.

In [2]:
```
# Import the libraries we use NetworkX for network analysis
import networkx as nx
import osmnx as ox

# Define the polygon for the admin boundary the 1st district of Budapest
admin_district = ox.geocode_to_gdf('1st district, Budapest')
admin_poly = admin_district.geometry.values[0]

# Download the road network for driving within the administrative boundary
G = ox.graph_from_polygon(admin_poly, network_type='drive')

# Print the number of nodes and edges in the road network
print("Number of nodes in the original graph:", G.number_of_nodes())
print("Number of edges in the original graph:", G.number_of_edges())
```

```
Number of nodes in the original graph: 397
Number of edges in the original graph: 781
```

In [3]:
```
# Convert the network graph to GeoDataFrames for nodes and edges
nodes, edges = ox.graph_to_gdfs(G)

# Print the number of nodes and edges in the road network
print("Number of nodes in the node table:", len(nodes))
print("Number of edges in the edge table:", len(edges))
```

```
Number of nodes in the node table: 397
Number of edges in the edge table: 781
```

In [4]:
```
# Create an empty graph
G = nx.MultiGraph()

# Add edges to the graph
for idx, row in edges.iterrows():
    G.add_edge(idx[0], idx[1], weight=row['length'])
```

```python
# Add nodes with location data to the graph
for idx, row in nodes.iterrows():
    G.add_node(idx, y=row['y'], x=row['x'])

# Print the number of nodes and edges in the graph
print("Number of nodes in the reconstructed graph:", G.number_of_nodes())
print("Number of edges in the reconstructed graph:", G.number_of_edges())
```

```
Number of nodes in the reconstructed graph: 397
Number of edges in the reconstructed graph: 781
```

In [5]:

```python
# Convert node attributes to a list
nodes_dict = list(G.nodes(data=True))
print(nodes_dict[0:5])
print()

# Convert edge attributes to a list
edges_dict = list(G.edges(data=True))
print(edges_dict[0:5])
```

```
[(35500835, {'y': 47.490117, 'x': 19.0456214}), (1676318037, {'y': 47.48992
39, 'x': 19.0465671}), (1676318015, {'y': 47.489206, 'x': 19.0468401}), (35
523878, {'y': 47.4913026, 'x': 19.0456141}), (10921844764, {'y': 47.491216
1, 'x': 19.045178})]

[(35500835, 1676318037, {'weight': 77.71969084118001}), (35500835, 16763180
15, {'weight': 139.1276873914375}), (35500835, 1676318029, {'weight': 723.2
186193856319}), (35500835, 1676318118, {'weight': 71.40030844416957}), (167
6318037, 1676318015, {'weight': 103.96067211518923})]
```

In this section, we provided an overview of how to create a spatial network by converting the node and edge GeoDataFrame tables, initially exported from a road network graph, back into the road network graph format while preserving all nodes and links. In addition to tracking the network size across processing steps, we confirmed that all attributes are added to the spatial network by printing the lists of nodes and edges.

83. Visualizing Spatial Networks

Visualizing spatial networks is an essential step in geospatial analysis, allowing us to manually observe and interpret the structure, connectivity, and spatial distribution of network elements such as roads and intersections. By creating visualizations of spatial networks, we can gain valuable insights into various use cases, formulate model hypotheses, and select modeling and analytical approaches accordingly.

In this section, we will demonstrate how to visualize a spatial network. First, we will use the built-in visualization routine in `osmnx`, then the more customizable plotting toolkit in `matplotlib`, setting specific plotting parameters, such as the edge attribute used for coloring (`length`), the colormap, or the line style. For both visuals, we will create network graphs, highlighting nodes and edges to showcase the overall structure and connectivity of the network of the 1st district of Budapest. As in the previous examples,

the code snippet below is easily adaptable to any other place name or polygon geometry.

In [6]:
```
# Import osmnx and matplotlib
import osmnx as ox
import matplotlib.pyplot as plt

# Define the polygon for the admin boundary the 1st district of Budapest
admin_district = ox.geocode_to_gdf('1st district, Budapest')
admin_poly = admin_district.geometry.values[0]

# Download the road network for driving within the administrative boundary
G = ox.graph_from_polygon(admin_poly, network_type='drive')

# Plot the network using OSMnx
fig, ax = ox.plot_graph(G,
                        node_size=10,
                        node_color='blue',
                        edge_color='gray',
                        bgcolor='white',
                        show=False,
                        close=False)

# Show the road network
ax.set_title('Spatial Network Visualization with OSMnx', fontsize=16)
plt.show()
```

Visualizing spatial networks with `osmnx` and `matplotlib` provides a clear, detailed depiction of the network's structure and connectivity. For example, we can observe a concentric pattern of lines representing the Buda Hill and the Castle District. In the second figure, the varying lengths of drivable road segments are highlighted by different colors. Such maps and road network structures are invaluable for urban planners and transportation engineers, providing insights into the local context and constraints defined by the city's inherent structure.

84. Calculating Shortest Paths

Calculating the shortest path in a spatial network is a fundamental task in areas such as transportation, route planning, and navigation. By identifying the most efficient route

between two points, we can optimize travel times, reduce congestion, and improve overall network efficiency. Using the previously introduced NetworkX library for graph analytics, we can apply graph-based algorithms to compute the shortest path in a spatial network.

In this section, we will demonstrate how to calculate the shortest path between two randomly chosen nodes in the example road network from the 1st district of Budapest. We will use the `shortest_path` function from NetworkX, specifying the road segment lengths as weights to find the most efficient route and output the shortest path expressed by a list of connected nodes, each denoted by its OSM ID.

In [8]:
```
# Necessary library imports
import networkx as nx
import random


# Define the polygon for the 1st district of Budapest
admin_district = ox.geocode_to_gdf('1st district, Budapest')
admin_poly = admin_district.geometry.values[0]

# Download the road network for driving within the admin boundary
G = ox.graph_from_polygon(admin_poly, network_type='drive')

# Convert the GeoDataFrame nodes to a list of unique node indices
nodes_l = list(set(nodes.index))

# Define the origin and destination nodes
origin = random.choice(nodes_l)
destination = random.choice(nodes_l)
print("Randomly picked origin intersection: ", origin)
print("Randomly picked destination intersection: ", destination)

# Calculate the shortest path
shortest_path = nx.shortest_path(G,
                                 source=origin,
                                 target=destination,
                                 weight='weight')
print("\nShortest path:", shortest_path)

# Calculate the total length of the shortest path
total_length = 0
for i in range(len(shortest_path) - 1):
    total_length += G[shortest_path[i]][shortest_path[i + 1]][0]['length']

print("\nTotal length of the shortest path:", len(shortest_path), "nodes")
print("Total length of the shortest path:", total_length, "meters")
```

```
Randomly picked origin intersection:  259020569
Randomly picked destination intersection:  277367628

Shortest path: [259020569, 10997076309, 3547379777, 3547379779, 3547379902,
4485192677, 2579836429, 259023754, 268193589, 2720958038, 259023613, 153420
899, 277166845, 153420889, 4886199474, 1692096447, 1692096443, 1692096442,
1692096441, 1692096435, 1692096434, 277888706, 277369229, 277367636, 277367
628]

Total length of the shortest path: 25 nodes
Total length of the shortest path: 2640.856272863735 meters
```